Assignment 2

Fernando Melo Filho
fmelofilho2950@floridapoly.edu

CAP 5410 — Computer Vision
Assignment 2

Instructor: Dr. Muhammad Abid

Image Filtering: Box, Sobel and Gaussian Filters

Department of Computer Science
Florida Polytechnic University, Florida, USA

September 2026

CONTENTS

CONTENTS ii

LIST OF FIGURES iv

LIST OF TABLES v

1 INTRODUCTION 1

1.1 OBJECTIVES 1

1.2 BACKGROUND 1

1.2.1 Box Filter 2

1.2.2 Sobel Filter 2

1.2.3 Gaussian Filter 2

1.3 REPORT ORGANIZATION 3

2 METHODOLOGY AND IMPLEMENTATION 4

2.1 DEVELOPMENT ENVIRONMENT 4

2.2 MANUAL CONVOLUTION 5

2.3 EXERCISE 1 — BOX FILTER WITHOUT OPENCV 5

2.4 EXERCISE 2 — BOX FILTER WITH OPENCV 6

2.5 EXERCISES 3 AND 4 — SOBEL X AND SOBEL Y WITHOUT OPENCV 6

2.6 EXERCISE 5 — SOBEL X AND Y WITHOUT OPENCV 7

2.7 EXERCISE 6 — SOBEL X AND Y WITH OPENCV 7

2.8 EXERCISE 7 — GAUSSIAN FILTER WITH OPENCV 8

2.9 TESTING THE MANUAL FILTERS 8

3 RESULTS 9

3.1 INPUT IMAGE 9

3.2 EXERCISE 1 — BOX FILTER WITHOUT OPENCV 10

3.3 EXERCISE 2 — BOX FILTER WITH OPENCV 11

3.4 EXERCISE 3 — SOBEL X WITHOUT OPENCV 11

3.5 EXERCISE 4 — SOBEL Y WITHOUT OPENCV 12
3.6 EXERCISE 5 — SOBEL X AND Y WITHOUT OPENCV 12
3.7 EXERCISE 6 — SOBEL X AND Y WITH OPENCV 13
3.8 EXERCISE 7 — GAUSSIAN FILTER WITH OPENCV 13

4 DISCUSSION AND CONCLUSION 15
4.1 DISCUSSION 15
4.2 CHALLENGES AND LIMITATIONS 15
4.3 CONCLUSION 16

REFERENCES 17

APPENDIX A SOURCE CODE 18

LIST OF FIGURES

Figure 3.1 Input image. 9
Figure 3.2 Box filter without OpenCV. 10
Figure 3.3 Box filter without OpenCV (zoom). 10
Figure 3.4 Box filter with OpenCV. 11
Figure 3.5 Sobel X without OpenCV. 11
Figure 3.6 Sobel Y without OpenCV. 12
Figure 3.7 Sobel X and Y without OpenCV. 12
Figure 3.8 Sobel X and Y with OpenCV. 13
Figure 3.9 Gaussian filter with OpenCV. 13
Figure 3.10 Box filter vs. Gaussian filter (zoom). 14

LIST OF TABLES

Table 2.1 Software environment. 4
Table 2.2 Requirements traceability. 8

1 INTRODUCTION

Filtering is one of the most common operations in image processing. It is used to remove noise, to smooth an image before further analysis, and to find edges, which are the starting point of many computer vision tasks. In this assignment I implemented three classic filters — the box filter, the Sobel filter, and the Gaussian filter — both by hand and with OpenCV, and applied each of them to the same input image with 3×3 and 5×5 kernels.

1.1 OBJECTIVES

The assignment requires implementing the following filters, each with a 3×3 and a 5×5 kernel:

1. Box filter, written without OpenCV.
2. Box filter, using OpenCV.
3. Sobel filter for X-axis edges, written without OpenCV.
4. Sobel filter for Y-axis edges, written without OpenCV.
5. Sobel filter combining X-axis and Y-axis edges, written without OpenCV.
6. Sobel filter combining X-axis and Y-axis edges, using OpenCV.
7. Gaussian filter, using OpenCV.

1.2 BACKGROUND

In Assignment 1 we worked with point operators, where each output pixel depends only on the matching input pixel. Filters are *neighborhood operators*: each output pixel is computed from a small window of pixels around it (Szeliski, 2022). The window is weighted by a small matrix called the *kernel* h, and the operation is written as

$$g(x, y) = \sum_{i=-r}^{r} \sum_{j=-r}^{r} h(i, j)\, f(x + i,\, y + j) \tag{1.1}$$

where f is the input image, g is the output image, and the kernel has size $k \times k$ with $r = \lfloor k/2 \rfloor$. The kernel "slides" over the image, and at every position the pixels under it are multiplied by the kernel weights and summed. The filters in this assignment only differ in the values of h.

1.2.1 Box Filter

The box (mean) filter replaces every pixel with the average of its $k \times k$ neighborhood, so all weights are equal to $1/k^2$ (Gonzalez and Woods, 2018):

$$h_{3\times3} = \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \tag{1.2}$$

Averaging reduces noise, but it also blurs edges and fine detail. A larger kernel averages more pixels and therefore blurs more.

1.2.2 Sobel Filter

The Sobel filter estimates the image gradient, i.e. how fast the intensity changes in each direction. Its kernel is the product of a smoothing vector and a derivative vector (Szeliski, 2022):

$$K_x = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \begin{bmatrix} -1 & 0 & 1 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \qquad K_y = K_x^T \tag{1.3}$$

K_x gives the horizontal gradient G_x, which is strong on vertical edges; K_y gives the vertical gradient G_y, which is strong on horizontal edges. The 5×5 version is built the same way from the vectors $[1, 4, 6, 4, 1]$ (smoothing) and $[-1, -2, 0, 2, 1]$ (derivative). The two directions are combined into the gradient magnitude

$$G = \sqrt{G_x^2 + G_y^2} \tag{1.4}$$

which responds to edges in any direction.

1.2.3 Gaussian Filter

The Gaussian filter is also a smoothing filter, but instead of equal weights it uses weights that follow a Gaussian (bell-shaped) curve, so the center pixel counts the most and far pixels count less (Gonzalez and Woods, 2018). With $\sigma = 0$, OpenCV derives σ from the kernel size; the resulting 1D weights are $[0.25, 0.5, 0.25]$ for 3×3 and $[0.0625, 0.25, 0.375, 0.25, 0.0625]$ for 5×5.

1.3 REPORT ORGANIZATION

The remainder of this report is organized as follows. Chapter 2 describes the development environment and the code for each of the seven exercises. Chapter 3 presents the output of every filter on the input image. Chapter 4 discusses the results and the challenges I faced, and concludes the report.

2 METHODOLOGY AND IMPLEMENTATION

I wrote all filters in Python. The functions live in a single module, `filters.py`, and the Jupyter notebook `notebooks/assignment2.ipynb` imports them, applies them to the input image, and saves every figure shown in Chapter 3. The manual filters use only NumPy (Harris et al., 2020); OpenCV (Bradski, 2000) is used only for the exercises that allow it, for loading the image, and as a reference to check my own code against.

2.1 DEVELOPMENT ENVIRONMENT

Table 2.1. Software environment used to develop and run the program.

Component	Version	Purpose
Python	3.13.5	Programming language
NumPy	2.5.3	Arrays, padding and the manual filters
OpenCV (opencv-python)	5.0.0.93	Image loading and the built-in filters (`blur`, `Sobel`, `GaussianBlur`)
Matplotlib	3.11.2	Displaying and saving the result figures (Hunter, 2007)
Jupyter / pytest	1.1.1 / 9.1.1	Running the exercises and testing the manual filters

The input image, `dog.bmp`, is a 410×361 color image. I load it once in grayscale, since all filters work on a single channel.

2.2 MANUAL CONVOLUTION

All the manual filters (Exercises 1, 3, 4 and 5) share one function, `convolve_manual` (Listing 2.1), which implements Equation 1.1. First it pads the image by r pixels on every side using reflection (`np.pad` with `mode="reflect"`), so that pixels on the border also have a full neighborhood. This is the same border mode OpenCV uses by default (`BORDER_REFLECT_101`) (OpenCV Team, 2026). Then it loops over every pixel, cuts out the $k \times k$ window, multiplies it element-wise by the kernel, and stores the sum. The work is done in `float64`, because the sums can be larger than 255 or negative.

```
def convolve_manual(image, kernel):
    """Slide `kernel` over a grayscale `image` and return a float64 result.

    Every output pixel is the sum of the k x k neighborhood multiplied
    element-wise by the kernel (correlation, the same convention used by
    OpenCV's filter2D/Sobel). The image is padded by reflection so that
    border pixels also have a full neighborhood.
    """
    k = kernel.shape[0]
    r = k // 2
    padded = np.pad(image.astype(np.float64), r, mode="reflect")
    out = np.zeros(image.shape, dtype=np.float64)
    rows, cols = image.shape
    for y in range(rows):
        for x in range(cols):
            window = padded[y:y + k, x:x + k]
            out[y, x] = np.sum(window * kernel)
    return out
```

Listing 2.1. Manual convolution used by all hand-written filters (`filters.py`).

2.3 EXERCISE 1 — BOX FILTER WITHOUT OPENCV

The box filter builds a $k \times k$ kernel of ones divided by k^2 (Equation 1.2) and passes it to `convolve_manual`. The result is rounded and clipped back to `uint8` so it can be displayed and compared with OpenCV (Listing 2.2). The same function handles both kernel sizes: `box_filter_manual(img, 3)` and `box_filter_manual(img, 5)`.

```
def box_filter_manual(image, k):
    """k x k box (mean) filter: every pixel becomes the average of its
       neighborhood."""
    kernel = np.ones((k, k), dtype=np.float64) / (k * k)
    out = convolve_manual(image, kernel)
    return np.clip(np.round(out), 0, 255).astype(np.uint8)
```

Listing 2.2. Manual box filter (`filters.py`).

2.4 EXERCISE 2 — BOX FILTER WITH OPENCV

With OpenCV, the box filter is a single call to `cv2.blur` (OpenCV Team, 2026) (Listing 2.3).

```
def box_filter_cv(image, k):
    return cv2.blur(image, (k, k))
```

Listing 2.3. Box filter with OpenCV (`filters.py`).

2.5 EXERCISES 3 AND 4 — SOBEL X AND SOBEL Y WITHOUT OPENCV

`sobel_kernels` builds K_x as the outer product of the smoothing and derivative vectors from Section 1.2, and K_y as its transpose (Listing 2.4). For $k = 5$ this gives the standard 5×5 Sobel kernel:

$$K_{x,5\times5} = \begin{bmatrix} -1 & -2 & 0 & 2 & 1 \\ -4 & -8 & 0 & 8 & 4 \\ -6 & -12 & 0 & 12 & 6 \\ -4 & -8 & 0 & 8 & 4 \\ -1 & -2 & 0 & 2 & 1 \end{bmatrix} \tag{2.1}$$

```
def sobel_kernels(k):
    """Return the (Kx, Ky) Sobel kernels for k = 3 or k = 5.

    Each Sobel kernel is the outer product of a smoothing vector and a
    derivative vector. Ky is the transpose of Kx.
    """
    if k == 3:
        smooth = np.array([1, 2, 1], dtype=np.float64)
        deriv = np.array([-1, 0, 1], dtype=np.float64)
    elif k == 5:
        smooth = np.array([1, 4, 6, 4, 1], dtype=np.float64)
        deriv = np.array([-1, -2, 0, 2, 1], dtype=np.float64)
    else:
        raise ValueError("kernel size must be 3 or 5")
    kx = np.outer(smooth, deriv)
    return kx, kx.T
```

Listing 2.4. Building the 3×3 and 5×5 Sobel kernels (`filters.py`).

The X and Y filters simply convolve the image with K_x or K_y (Listing 2.5). The output is kept as a signed `float64` array, because a gradient is negative when the intensity goes from bright to dark.

```
def sobel_x_manual(image, k):
    """Horizontal gradient Gx (responds to vertical edges). Signed float64
    ."""
    kx, _ = sobel_kernels(k)
    return convolve_manual(image, kx)


```

```
def sobel_y_manual(image, k):
    """Vertical gradient Gy (responds to horizontal edges). Signed float64."""
    _, ky = sobel_kernels(k)
    return convolve_manual(image, ky)
```

Listing 2.5. Manual Sobel X (Exercise 3) and Sobel Y (Exercise 4) filters (`filters.py`).

A gradient image cannot be shown directly, since its values go far outside $[0, 255]$ (for example, -470 to 465 for 3×3 and -6032 to 4994 for 5×5). To display it, I take the absolute value and scale it so the strongest edge becomes 255 (Listing 2.6). Scaling each image by its own maximum also makes the 3×3 and 5×5 results comparable side by side.

```
def to_display(gradient):
    """Map a signed gradient to 0-255 for display: |g| scaled by its maximum."""
    mag = np.abs(gradient)
    return (mag / mag.max() * 255).astype(np.uint8)
```

Listing 2.6. Mapping a gradient to 0–255 for display (`filters.py`).

2.6 EXERCISE 5 — SOBEL X AND Y WITHOUT OPENCV

The combined filter runs both manual Sobel filters and joins them with the gradient magnitude of Equation 1.4 (Listing 2.7).

```
def sobel_xy_manual(image, k):
    """Gradient magnitude sqrt(Gx^2 + Gy^2) from the manual Sobel filters."""
    gx = sobel_x_manual(image, k)
    gy = sobel_y_manual(image, k)
    return np.sqrt(gx ** 2 + gy ** 2)
```

Listing 2.7. Manual combined Sobel filter (`filters.py`).

2.7 EXERCISE 6 — SOBEL X AND Y WITH OPENCV

With OpenCV, `cv2.Sobel` computes each direction and `cv2.magnitude` combines them (OpenCV Team, 2026). I request a `CV_64F` output so negative gradients are not cut off (Listing 2.8).

```
def sobel_xy_cv(image, k):
    gx = cv2.Sobel(image, cv2.CV_64F, 1, 0, ksize=k)
    gy = cv2.Sobel(image, cv2.CV_64F, 0, 1, ksize=k)
    return cv2.magnitude(gx, gy)
```

Listing 2.8. Combined Sobel filter with OpenCV (`filters.py`).

2.8 EXERCISE 7 — GAUSSIAN FILTER WITH OPENCV

The Gaussian filter uses `cv2.GaussianBlur` with $\sigma = 0$, so OpenCV chooses σ from the kernel size (Listing 2.9).

```
def gaussian_cv(image, k):
    # sigma = 0 lets OpenCV derive sigma from the kernel size
    return cv2.GaussianBlur(image, (k, k), 0)
```

Listing 2.9. Gaussian filter with OpenCV (`filters.py`).

2.9 TESTING THE MANUAL FILTERS

To make sure the hand-written filters are correct, I wrote a small set of tests with pytest (`test_box_filter.py`). They check that the output has the same shape as the input, that a constant image stays constant after the box filter, that a sharp $0 \mid 255$ edge turns into the expected ramp $(85, 170)$, and that both the manual box filter and the manual Sobel filters match OpenCV. All five tests pass.

Table 2.2. Mapping of each exercise to its implementation and to the figure that shows its output.

#	Exercise	Implemented in	Evidence
1	Box filter (manual)	`box_filter_manual`	Figs. 3.2, 3.3
2	Box filter (OpenCV)	`box_filter_cv`	Fig. 3.4
3	Sobel X (manual)	`sobel_x_manual`	Fig. 3.5
4	Sobel Y (manual)	`sobel_y_manual`	Fig. 3.6
5	Sobel X + Y (manual)	`sobel_xy_manual`	Fig. 3.7
6	Sobel X + Y (OpenCV)	`sobel_xy_cv`	Fig. 3.8
7	Gaussian filter (OpenCV)	`gaussian_cv`	Figs. 3.9, 3.10

3 RESULTS

This chapter shows the output of every filter, taken from the executed notebook `notebooks/assignment2.ipynb`. Each figure shows the original image on the left, followed by the 3×3 and the 5×5 result.

3.1 INPUT IMAGE

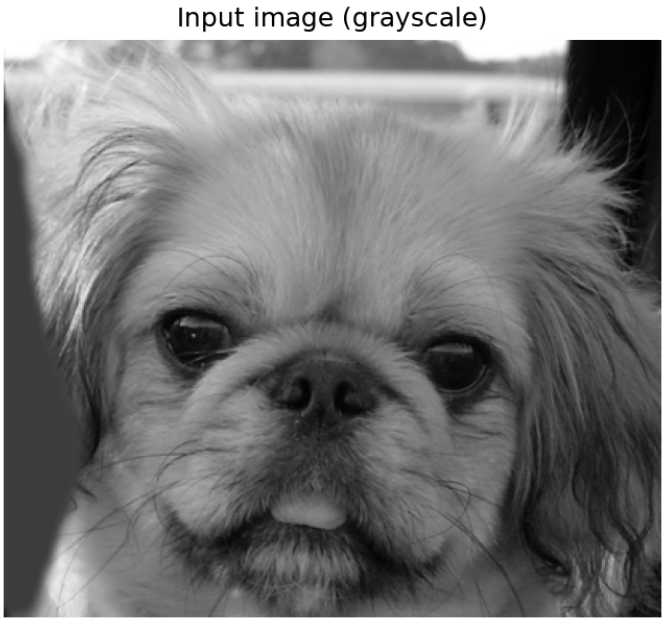


Figure 3.1. Input image `dog.bmp` (410×361), loaded in grayscale.

3.2 EXERCISE 1 — BOX FILTER WITHOUT OPENCV

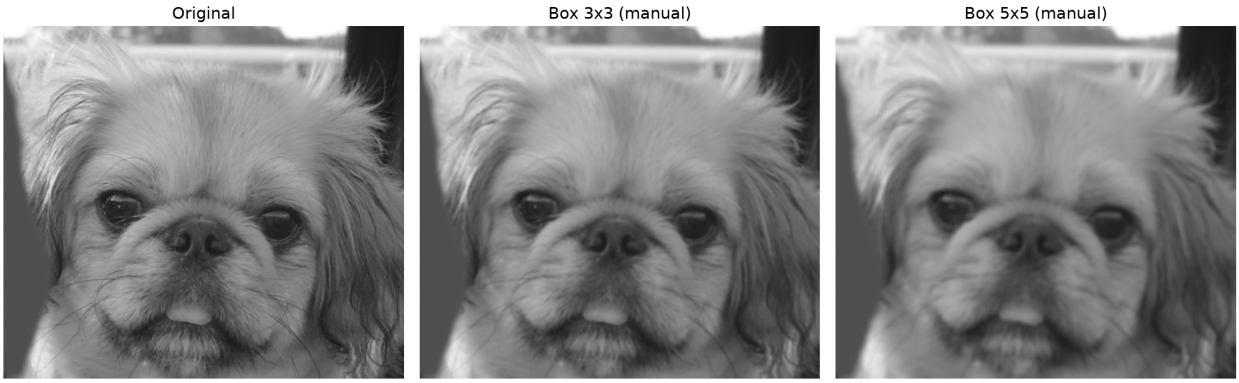


Figure 3.2. Manual box filter with 3×3 and 5×5 kernels.

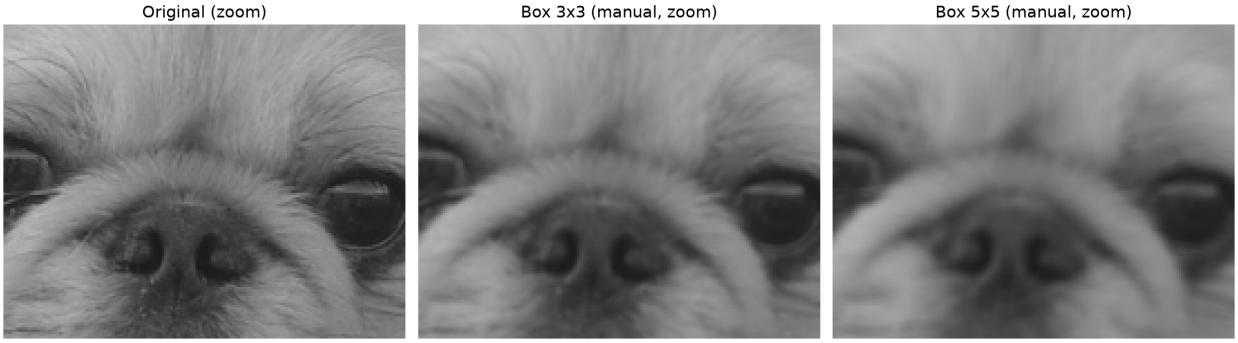


Figure 3.3. Same results zoomed in on the eyes and nose, where the blur is easier to see.

Observation. Both kernels blur the image; the fur, whiskers and the reflection in the eyes lose detail. The 5×5 kernel averages 25 pixels instead of 9, so it blurs noticeably more than the 3×3 kernel. The difference is hard to see at full size but clear in the zoomed view (Figure 3.3). Each manual run took about 0.3 s.

3.3 EXERCISE 2 — BOX FILTER WITH OPENCV

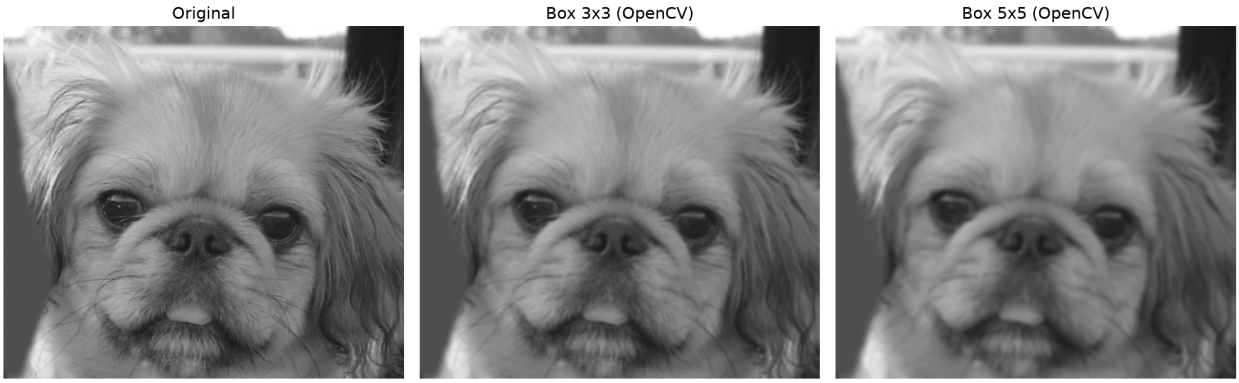


Figure 3.4. OpenCV box filter (`cv2.blur`) with 3×3 and 5×5 kernels.

Observation. The OpenCV output is identical to my manual implementation: the maximum pixel difference is 0 for both kernel sizes, including the borders, because both use reflection padding. The only difference is speed: `cv2.blur` runs in well under a millisecond (0.2–2 ms), while my Python loop takes about 0.3 s, which is hundreds of times slower.

3.4 EXERCISE 3 — SOBEL X WITHOUT OPENCV



Figure 3.5. Manual Sobel filter in the X direction ($|G_x|$), 3×3 and 5×5.

Observation. The X filter highlights vertical edges: the sides of the face, the vertical strands of fur on the ears, and the vertical post in the background at the top right. Horizontal structures, such as the bar at the top of the image and the mouth, almost disappear. The 3×3 result shows many

thin, fine edges (and more texture noise), while the 5×5 result gives thicker, smoother edges, because its kernel also smooths over a wider area.

3.5 EXERCISE 4 — SOBEL Y WITHOUT OPENCV



Figure 3.6. Manual Sobel filter in the Y direction ($|G_y|$), 3×3 and 5×5.

Observation. The Y filter is the opposite of the X filter: it highlights horizontal edges, such as the bar in the background, the top and bottom of the eyes, the nose and the mouth, while the vertical sides of the face become weak. Again, the 5×5 kernel produces thicker and less noisy edges than the 3×3 kernel.

3.6 EXERCISE 5 — SOBEL X AND Y WITHOUT OPENCV



Figure 3.7. Manual gradient magnitude $\sqrt{G_x^2 + G_y^2}$, 3×3 and 5×5.

Observation. Combining both directions gives a complete edge map: the outline of the eyes, nose, mouth and fur is visible no matter which direction the edge runs. The 3×3 magnitude is sharper and shows more fine texture; the 5×5 magnitude is smoother and keeps mostly the stronger edges.

3.7 EXERCISE 6 — SOBEL X AND Y WITH OPENCV



Figure 3.8. Gradient magnitude computed with `cv2.Sobel` and `cv2.magnitude`, 3×3 and 5×5.

Observation. The OpenCV result matches my manual implementation exactly (maximum difference 0.0 for both kernel sizes), which confirms that my 3×3 and 5×5 kernels and my border handling are the same as OpenCV's.

3.8 EXERCISE 7 — GAUSSIAN FILTER WITH OPENCV



Figure 3.9. Gaussian filter (`cv2.GaussianBlur`, $\sigma = 0$), 3×3 and 5×5.



Figure 3.10. Box filter and Gaussian filter with the same 5×5 kernel size, zoomed in on the eyes and nose.

Observation. The Gaussian filter also smooths the image, but with the same kernel size it blurs less than the box filter and keeps the edges more natural (Figure 3.10). This is because the center pixel has the largest weight and the pixels further away count less, while the box filter gives the same weight to every pixel in the window. As with the box filter, the 5×5 kernel blurs more than the 3×3 kernel.

4 DISCUSSION AND CONCLUSION

4.1 DISCUSSION

The main goal of this assignment was to write the filters myself and then compare them with the OpenCV built-in functions. After implementing the manual box and Sobel filters, I checked them against `cv2.blur` and `cv2.Sobel`, and the maximum difference was 0 for both kernel sizes (Sections 3.3 and 3.7). This showed me that OpenCV is not doing anything magical: it applies the same kernels, with the same reflection padding at the borders, that I wrote by hand. The big difference is speed. My Python loop takes about 0.3 s per image, while OpenCV finishes in under a millisecond, because it runs optimized C++ code instead of visiting every pixel from Python. Writing the filters myself also made it clear that the box, Sobel and Gaussian filters all use the same sliding-window operation, and that only the values inside the kernel decide whether the image gets smoothed or its edges get detected.

4.2 CHALLENGES AND LIMITATIONS

My biggest challenge was understanding the math behind the filters. At first I thought of the operation as a matrix multiplication, but a filter works differently: the $k \times k$ window of the image is multiplied by the kernel element by element (each pixel times the weight in the same position), and then all the products are added up. This is the same as a dot product between the window and the kernel written as two long vectors (Equation 1.1). The second part was understanding the sliding window: the kernel is centered on one pixel, produces one output value, and then moves one pixel to the right, row by row, until it has covered the whole image. Once this idea was clear, the code became simple, as in `convolve_manual` (Listing 2.1), where it is just two loops, a slice, and a sum. The sliding window also raised two practical problems. Pixels on the border do not have a full neighborhood, so I had to pad the image, and I used reflection to match OpenCV. The sums can also go above 255 or below 0, especially for the Sobel filter, so I did all the math in `float64` and only converted back to 0–255 at the end.

4.3 CONCLUSION

Filters are neighborhood operations: each output pixel is computed from a window of pixels around it, weighted by a kernel. In this assignment I implemented the box and Sobel filters by hand and used OpenCV for the box, Sobel and Gaussian filters, with 3×3 and 5×5 kernels. The smoothing filters removed detail, and the 5×5 kernels blurred more, with the Gaussian keeping edges better than the box filter. The Sobel filters found vertical edges, horizontal edges, or both combined, and my manual versions produced exactly the same results as OpenCV.

REFERENCES

Bradski, G. 2000. "The OpenCV library." *Dr. Dobb's Journal of Software Tools*.

Gonzalez, R. C. and Woods, R. E. 2018. *Digital Image Processing*. Pearson, 4th edition.

Harris, C. R. et al. 2020. "Array programming with NumPy." *Nature*, 585: 357–362.

Hunter, J. D. 2007. "Matplotlib: A 2D graphics environment." *Computing in Science & Engineering*, 9 (3): 90–95.

OpenCV Team 2026. "OpenCV image filtering documentation. Accessed September 2026.

Szeliski, R. 2022. *Computer Vision: Algorithms and Applications*. Springer, 2nd edition.

A SOURCE CODE

The complete contents of `filters.py` and `test_box_filter.py` are listed below; these are the same files submitted alongside this report, together with the notebook `notebooks/assignment2.ipynb` that runs every exercise.

A.1 FILTERS.PY

```
"""CAP 5410 - Assignment 2: image filters.

Manual (no OpenCV) implementations of the box filter and the Sobel
filters, plus the OpenCV versions used for comparison.
"""
import numpy as np
import cv2


# ---------------------------------------------------------------------
# Manual implementations (no OpenCV)
# ---------------------------------------------------------------------

def convolve_manual(image, kernel):
    """Slide `kernel` over a grayscale `image` and return a float64 result.

    Every output pixel is the sum of the k x k neighborhood multiplied
    element-wise by the kernel (correlation, the same convention used by
    OpenCV's filter2D/Sobel). The image is padded by reflection so that
    border pixels also have a full neighborhood.
    """
    k = kernel.shape[0]
    r = k // 2
    padded = np.pad(image.astype(np.float64), r, mode="reflect")
    out = np.zeros(image.shape, dtype=np.float64)
    rows, cols = image.shape
    for y in range(rows):
        for x in range(cols):
            window = padded[y:y + k, x:x + k]
            out[y, x] = np.sum(window * kernel)
```

```
    return out


def box_filter_manual(image, k):
    """k x k box (mean) filter: every pixel becomes the average of its neighborhood."""
    kernel = np.ones((k, k), dtype=np.float64) / (k * k)
    out = convolve_manual(image, kernel)
    return np.clip(np.round(out), 0, 255).astype(np.uint8)


def sobel_kernels(k):
    """Return the (Kx, Ky) Sobel kernels for k = 3 or k = 5.

    Each Sobel kernel is the outer product of a smoothing vector and a
    derivative vector. Ky is the transpose of Kx.
    """
    if k == 3:
        smooth = np.array([1, 2, 1], dtype=np.float64)
        deriv = np.array([-1, 0, 1], dtype=np.float64)
    elif k == 5:
        smooth = np.array([1, 4, 6, 4, 1], dtype=np.float64)
        deriv = np.array([-1, -2, 0, 2, 1], dtype=np.float64)
    else:
        raise ValueError("kernel size must be 3 or 5")
    kx = np.outer(smooth, deriv)
    return kx, kx.T


def sobel_x_manual(image, k):
    """Horizontal gradient Gx (responds to vertical edges). Signed float64."""
    kx, _ = sobel_kernels(k)
    return convolve_manual(image, kx)


def sobel_y_manual(image, k):
    """Vertical gradient Gy (responds to horizontal edges). Signed float64."""
    _, ky = sobel_kernels(k)
    return convolve_manual(image, ky)


def sobel_xy_manual(image, k):
    """Gradient magnitude sqrt(Gx^2 + Gy^2) from the manual Sobel filters."""
    gx = sobel_x_manual(image, k)
    gy = sobel_y_manual(image, k)
    return np.sqrt(gx ** 2 + gy ** 2)


# ---------------------------------------------------------------
# OpenCV implementations
# ---------------------------------------------------------------
```

```
81
82  def box_filter_cv(image, k):
83      return cv2.blur(image, (k, k))
84
85
86  def sobel_xy_cv(image, k):
87      gx = cv2.Sobel(image, cv2.CV_64F, 1, 0, ksize=k)
88      gy = cv2.Sobel(image, cv2.CV_64F, 0, 1, ksize=k)
89      return cv2.magnitude(gx, gy)
90
91
92  def gaussian_cv(image, k):
93      # sigma = 0 lets OpenCV derive sigma from the kernel size
94      return cv2.GaussianBlur(image, (k, k), 0)
95
96
97  # ---------------------------------------------------------------
98  # Display helper
99  # ---------------------------------------------------------------
100
101 def to_display(gradient):
102     """Map a signed gradient to 0-255 for display: |g| scaled by its
            maximum."""
103     mag = np.abs(gradient)
104     return (mag / mag.max() * 255).astype(np.uint8)
```

Listing A.1. Complete source code (`filters.py`).

A.2 TEST_BOX_FILTER.PY

```
1   import numpy as np
2   import cv2
3   from filters import box_filter_manual, sobel_x_manual, sobel_y_manual
4
5
6   def test_output_has_same_shape_as_input():
7       rng = np.random.default_rng(0)
8       img = rng.integers(0, 256, (20, 30), dtype=np.uint8)
9       for k in (3, 5):
10          assert box_filter_manual(img, k).shape == img.shape
11
12
13  def test_constant_image_stays_constant():
14      # Averaging identical values must give the same value back.
15      # Catches wrong divisors AND uint8 overflow while summing.
16      img = np.full((10, 10), 100, dtype=np.uint8)
17      for k in (3, 5):
18          out = box_filter_manual(img, k)
19          r = k // 2
20          assert np.allclose(out[r:-r, r:-r], 100)
21
```

```
22
23  def test_vertical_edge_becomes_ramp():
24      # The worked example from our chat: 0 | 255 edge -> 85, 170
25      img = np.array([[0, 0, 255, 255]] * 3, dtype=np.uint8)
26      out = box_filter_manual(img, 3)
27      assert np.isclose(out[1, 1], 85)
28      assert np.isclose(out[1, 2], 170)
29
30
31  def test_matches_opencv_in_interior():
32      # cv2 is used ONLY here, as a reference to check against.
33      rng = np.random.default_rng(1)
34      img = rng.integers(0, 256, (50, 60), dtype=np.uint8)
35      for k in (3, 5):
36          mine = box_filter_manual(img, k).astype(float)
37          ref = cv2.blur(img, (k, k)).astype(float)
38          r = k // 2
39          assert np.abs(mine[r:-r, r:-r] - ref[r:-r, r:-r]).max() <= 1
40
41
42  def test_sobel_matches_opencv():
43      # Same border handling (reflect) as OpenCV's default, so every pixel
            matches.
44      rng = np.random.default_rng(2)
45      img = rng.integers(0, 256, (40, 50), dtype=np.uint8)
46      for k in (3, 5):
47          assert np.allclose(sobel_x_manual(img, k), cv2.Sobel(img, cv2.
                CV_64F, 1, 0, ksize=k))
48          assert np.allclose(sobel_y_manual(img, k), cv2.Sobel(img, cv2.
                CV_64F, 0, 1, ksize=k))
```

Listing A.2. Tests for the manual filters (`test_box_filter.py`).